

Problem A. One-time guests

Input file: **standard input**
Output file: **standard output**
Time limit: 1 second
Memory limit: 256 megabytes

Jojo is watching a stream of lowercase Latin letters arrive one by one.

After every single letter that arrives, he wants to know the **first** letter of the stream so far that has occurred exactly once — “first” meaning the leftmost one among all letters that are still unique. If at that moment every letter seen so far has occurred more than once, the answer is -1 .

Help Jojo answer this after each arrival.

Input

The first line contains one integer T — the number of test cases ($1 \leq T \leq 200$).

Each test case is given on two lines. The first of them contains one integer N — the length of the stream ($1 \leq N \leq 500$). The second contains N lowercase Latin letters separated by single spaces — the letters in the order they arrive.

Output

For each test case print one line with N answers separated by single spaces: the i -th of them is the first non-repeating letter of the stream after the first i letters have arrived, or -1 if at that moment there is no such letter.

Examples

standard input	standard output
2 4 a a b c 3 a a c	a -1 b b a -1 c
1 6 a d b c a a	a a a a d d

Note

In the first test case the stream is a, a, b, c :

- after “a” the only letter is a and it is unique — the answer is a ;
- after the second “a” the letter a has occurred twice, and there is nothing else — the answer is -1 ;
- after “b” the unique letters are b only — the answer is b ;
- after “c” the unique letters are b and c , and b came first — the answer is b .

In the second test case the stream is a, d, b, c, a, a , and the answers are a, a, a, a, d, d .

Problem B. Kuanyshbek

Input file: **standard input**
Output file: **standard output**
Time limit: 1 second
Memory limit: 256 megabytes

Kuanyshbek has been studying how multithreading works in operating systems. He wrote a program that starts a single thread which writes to a single file. Being careless, he actually launched **two** threads with different parameters writing into the same file. The OS lets only one thread write at a time, so the two threads took turns: the first record came from thread one, the second from thread two, the third from thread one again, and so on.

Kuanyshbek only needs the data of the first thread. Help him throw away everything the second thread wrote — that is, every element standing at an even position, counting from 1.

Solve this problem with a linked list: build the list, then delete every second node.

Input

The first line contains one integer N — how many records ended up in the file ($1 \leq N \leq 99$).

The second line contains N integers separated by single spaces — the records in the order they were written ($1 \leq a_i \leq 10^6$).

Output

Print the values that are left in the list after every second one has been deleted, separated by single spaces — that is, the records at positions 1, 3, 5, ...

Example

standard input	standard output
5 1 2 3 4 5	1 3 5

Note

In the example the records at positions 2 and 4 — the values 2 and 4 — came from the second thread and are deleted. The values 1, 3 and 5 are left.

Problem C. Database

Input file: **standard input**
Output file: **standard output**
Time limit: 1 second
Memory limit: 256 megabytes

The KBTU database has crashed and the list of all students came back damaged: some names got duplicated. The way the crash worked, all copies of the same name always ended up in **consecutive** positions. For instance, the name “Dana” may occupy positions 2, 3, 4, but it can never occupy positions 2, 3, 5, because those are not consecutive.

Help the teachers get rid of the duplicates: keep only the first record of every group of equal neighbours, without changing the order of the list.

Input

The first line contains one integer N — the number of records ($1 \leq N \leq 10^5$).

Each of the next N lines contains one name: a non-empty string of at most 20 Latin letters. Both uppercase and lowercase letters may occur, and they are considered different: “Aa” and “aA” are two different names.

All records of the same name are guaranteed to occupy consecutive positions.

Output

On the first line print one integer — how many students are left after the duplicates are removed.

Then print those names, one per line, **in the same order in which they appear in the input**.

Examples

standard input	standard output
5 Alice Dana Dana Dana Bob	3 Alice Dana Bob
2 wow kek	2 wow kek
3 kek wow wow	2 kek wow

Note

In the first example “Dana” occupies positions 2, 3, 4, so two of the three records are dropped and three students remain: Alice, Dana and Bob — in exactly that order.

In the second example there is nothing to remove.

In the third example the two copies of “wow” stand next to each other, so one of them is dropped.

Problem D. Reverse the Linked List

Input file: standard input
Output file: standard output
Time limit: 1 second
Memory limit: 256 megabytes

Aisha is writing a music player. The playlist is stored as a **singly linked list**: every song knows only which song comes after it, and nothing about the song before it.

She has just added a **Play in reverse** button. The player cannot step backwards — it can only follow *next* pointers — so pressing the button has to rebuild the list itself, turning every *next* pointer around.

Given a playlist, print it the way the player will see it after the button is pressed.

Input

The first line contains one integer n — the number of songs in the playlist ($1 \leq n \leq 2 \cdot 10^5$).

The second line contains n integers $a_1, a_2, \dots, a_n$ — the identifiers of the songs, listed from the head of the list to its tail ($-10^9 \leq a_i \leq 10^9$).

Output

Print n integers separated by single spaces — the identifiers of the songs from the head of the reversed list to its tail.

Examples

standard input	standard output
5 1 2 3 4 5	5 4 3 2 1
1 -7	-7

Note

In the first example the list $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5$ becomes $5 \rightarrow 4 \rightarrow 3 \rightarrow 2 \rightarrow 1$.

In the second example the playlist holds a single song, so reversing it changes nothing.

Problem E. Delete the Middle Node

Input file: **standard input**
Output file: **standard output**
Time limit: 1 second
Memory limit: 256 megabytes

The scoreboard of a relay race is kept as a **singly linked list** of runners, in the order they crossed the finish line.

One of the runners turned out to be a pacemaker who was never part of the competition, and by the rules of this race the pacemaker always finishes exactly in the middle. If the scoreboard holds n runners, numbered $0, 1, \dots, n - 1$ starting from the head of the list, the pacemaker is the runner with number $\lfloor n/2 \rfloor$.

Remove that runner from the list and print what is left of the scoreboard.

Input

The first line contains one integer n — the number of runners on the scoreboard ($1 \leq n \leq 2 \cdot 10^5$).

The second line contains n integers $a_1, a_2, \dots, a_n$ — the identifiers of the runners, listed from the head of the list to its tail ($-10^9 \leq a_i \leq 10^9$).

Output

Print the $n - 1$ remaining identifiers separated by single spaces, from the head of the list to its tail.

If $n = 1$, the list becomes empty — print an empty line.

Examples

standard input	standard output
5 10 20 30 40 50	10 20 40 50
4 1 2 3 4	1 2 4
1 42	

Note

In the first example $n = 5$, so the runner to remove is the one with number $\lfloor 5/2 \rfloor = 2$, that is the value 30.

In the second example $n = 4$ and $\lfloor 4/2 \rfloor = 2$, so the value 3 is removed. Note that for an even length the node to remove is the **second** of the two middle ones.

In the third example the scoreboard holds a single runner, who is removed, and the output is empty.

Problem F. Merge Two Sorted Lists

Input file: **standard input**
Output file: **standard output**
Time limit: 1 second
Memory limit: 256 megabytes

Two ticket offices at a railway station keep their own waiting lists. Each list is a **singly linked list** of ticket numbers, and inside each list the numbers already go in non-decreasing order.

The offices are being merged into one. The new waiting list must contain every ticket from both lists and must again be in non-decreasing order.

The station manager refuses to pay for extra memory: the merged list has to be built out of the nodes that already exist, only by re-linking their *next* pointers. No new node may be created.

Input

The first line describes the first list. It starts with the length n ($0 \leq n \leq 10^5$), followed by n integers in non-decreasing order — the ticket numbers from the head of the list to its tail ($-10^9 \leq a_i \leq 10^9$). If $n = 0$, the line contains just the single number 0.

The second line describes the second list in exactly the same way: its length m ($0 \leq m \leq 10^5$) followed by m integers in non-decreasing order ($-10^9 \leq b_i \leq 10^9$).

Output

Print $n + m$ integers in non-decreasing order, separated by single spaces — the merged waiting list from its head to its tail.

If both lists are empty, print an empty line.

Examples

standard input	standard output
3 1 3 5 3 2 4 6	1 2 3 4 5 6
0 3 2 4 6	2 4 6
0 0	

Note

In the first example the two lists interleave completely.

In the second example the first list is empty, so the answer is the second list unchanged. Watch this case: a solution that starts by reading $a \rightarrow val$ without first checking that the list is non-empty will crash here.

In the third example both lists are empty and the output is empty.

Problem G. Jonathan the Poet

Input file: `standard input`
Output file: `standard output`
Time limit: 1 second
Memory limit: 256 megabytes

Jonathan the Poet has just finished a new poem, but something about the rhyme feels off. He is convinced that shifting the poem cyclically by K positions will fix it.

The poem is stored as a **singly linked list** of words: every word knows only which word comes after it. A cyclic shift by K moves the first K words to the very end, keeping their order, so the poem

$$w_1 \ w_2 \ \dots \ w_K \ w_{K+1} \ \dots \ w_N$$

turns into

$$w_{K+1} \ \dots \ w_N \ w_1 \ w_2 \ \dots \ w_K.$$

Jonathan is tired. Help him and print the shifted poem.

Input

The first line contains two integers N and K — the number of words in the poem and the size of the shift ($1 \leq K < N \leq 10^5$).

The second line contains N words separated by single spaces. Every word is a non-empty string of lowercase Latin letters, and the total length of all words does not exceed $3 \cdot 10^5$.

Output

Print the N words of the shifted poem, separated by single spaces.

Examples

standard input	standard output
5 2 the show must go on	must go on the show
5 3 another one bites the dust	the dust another one bites

Note

In the first example $N = 5$ and $K = 2$: the first two words, “the” and “show”, move to the end.

In the second example $K = 3$, so “another one bites” moves to the end and the poem starts with “the dust”.

Problem H. Ragnarok

Input file: **standard input**
Output file: **standard output**
Time limit: 1 second
Memory limit: 256 megabytes

Thor keeps a chronicle of the days before Ragnarok. Every day is written down as a number: a good day is a positive number, a bad day is a negative one. The chronicle is stored as a **singly linked list** — each day only knows which day comes after it.

Thor wants to find the best stretch of the chronicle: a non-empty run of **consecutive** days whose numbers add up to the largest possible sum. Print that sum.

Input

The first line contains one integer n — the number of days in the chronicle ($1 \leq n \leq 100$).

The second line contains n integers $a_1, a_2, \dots, a_n$ — the days in the order they are stored in the list, from its head to its tail ($-10^4 \leq a_i \leq 10^4$).

Output

Print one integer — the largest sum over all non-empty runs of consecutive days.

Examples

standard input	standard output
5 1 2 -1 4 5	11
3 -1 -1 -1	-1
10 5 1 2 -10 5 3 9 -5 10 10	32

Note

In the first example the best run is the whole chronicle: $1 + 2 - 1 + 4 + 5 = 11$.

In the second example every day is bad, so the best you can do is take a single day: -1 . Note that the run must be non-empty — the answer is never 0 here.

In the third example the best run is $5 + 3 + 9 - 5 + 10 + 10 = 32$.

Problem I. Doubly linked list

Input file: **standard input**
Output file: **standard output**
Time limit: 1 second
Memory limit: 256 megabytes

Implement a **doubly linked list** that stores the titles of books, and run a sequence of commands on it. The list must support the following commands:

- **add_front <title>** — add a book to the beginning of the list. Print **ok**.
- **add_back <title>** — add a book to the end of the list. Print **ok**.
- **erase_front** — remove the first book. Print the title of the removed book.
- **erase_back** — remove the last book. Print the title of the removed book.
- **front** — print the title of the first book, without removing it.
- **back** — print the title of the last book, without removing it.
- **clear** — remove every book from the list. Print **ok**.
- **exit** — print **goodbye** and stop.

If the list is **empty** when **erase_front**, **erase_back**, **front** or **back** is called, there is nothing to report — print **error** instead and leave the list as it is. The commands **clear** and **exit** always work, even on an empty list.

Input

Each line of the input contains one command from the list above. The commands **add_front** and **add_back** are followed on the same line, after a single space, by the title of the book: a non-empty string of at most 20 characters, each of them a Latin letter, a digit or an underscore.

The last command of the input is always **exit**, and it is the only occurrence of **exit**. The total number of commands does not exceed 10^5 .

Output

For each command print its answer on a separate line, as described above.

Example

standard input	standard output
add_front Harry_Potter	ok
add_back Light	ok
erase_front	Harry_Potter
erase_back	Light
erase_front	error
add_front Happy	ok
back	Happy
add_back Autumn	ok
add_front Alchemy	ok
clear	ok
front	error
exit	goodbye

Note

Let us follow the first example step by step:

- `add_front Harry_Potter` — the list becomes `[Harry_Potter]`, print `ok`;
- `add_back Light` — `[Harry_Potter, Light]`, print `ok`;
- `erase_front` — removes `Harry_Potter`, print `Harry_Potter`;
- `erase_back` — removes `Light`, print `Light`;
- `erase_front` — the list is already empty, print `error`;
- `add_front Happy` — `[Happy]`, print `ok`;
- `back` — print `Happy`;
- `add_back Autumn`, `add_front Alchemy` — `[Alchemy, Happy, Autumn]`, print `ok` twice;
- `clear` — the list becomes empty, print `ok`;
- `front` — the list is empty, print `error`;
- `exit` — print `goodbye`.

Problem J. 195823. Zoro and Seven Sword Style.

Input file: standard input
Output file: standard output
Time limit: 1.5 seconds
Memory limit: 256 megabytes

THE CODE TEMPLATE IS IN THE NOTE BELOW.

Zoro got lost again, this time in the maze. Walking along a random corridor, he stumbles upon a mysterious door, which says that this is the exit from the maze. The door mechanism works on specific functions for the linked list. But due to the fact that the door is very old, the functions have been erased. Zoro discovered an ancient stone panegyph nearby, which lists about each function:

1. inserts — add a node on position p.
2. remove — remove the node from position p.
3. print — print all values of list separated by a space.
4. replace — move the node from position p1 and to position p2. Position p2 is considered at the moment after its removal.
5. reverse — reverse the entire list.
6. cyclic_left — do a cyclic shift to the left x times.
7. cyclic_right — do a cyclic shift to the right x times.

Also, there are indicated the commands that need to be executed in order for the door to open. It is known that each command calls a specific function. Help Zoro to restore functions.

Input

Each line of input starts with integer which indicates command:

- If command 0, exit the program.
- If command 1, then the same line of input contains numbers x ($0 \leq x \leq 10^6$) and p ($0 \leq p$). Add a new node with value x to the position p. It is guaranteed that p does not exceed the length of the list.
- If command 2, then the same line of input contains number p ($0 \leq p$). Delete the node from position p. It is guaranteed that p is less than the length of the list.
- If command 3, print the whole list. Print -1 if list is empty.
- If command 4, then the same line of input contains numbers p1 and p2 ($0 \leq p1, p2$). Move node from position p1 to position p2. Position p2 is counted from the moment when we have already retrieved the node from position p1. It is guaranteed that p1 and p2 are less than the length of the list.
- if command 5, reverse whole list.
- If command 6, then the same line of input contains number x. Make left cyclic shift x ($0 \leq x$) times. It is guaranteed that x is less than the length of the list.
- If command 7, then the same line of input contains number x. Make right cyclic shift x ($0 \leq x$) times. It is guaranteed that x is less than the length of the list.

Subtasks

1. (20%) Implement each function in $O(N^2)$ or faster.
2. (20%) Implement functions inserts, remove, print and replace in $O(N)$.
3. (20%) Implement functions inserts, remove, print and reverse in $O(N)$.
4. (20%) Implement functions inserts, remove, print, cyclic_left and cyclic_right in $O(N)$.
5. (20%) Implement all of the functions in $O(N)$.

Output

For each `print` command, print all values of the list separated by single spaces.

Note

Each function except `print` must return the head of the linked list.

The statement above fully describes the input and output format, so you can write the whole program yourself. If you prefer to start from a ready-made template with the reading loop already written, take the one for your language:

C++ : <https://pastebin.com/BAG1n8Kp>

Python : <https://pastebin.com/9mwkZnEh>

Java : <https://pastebin.com/jfhpYWYR>

Just leave it to luck

— Roronoa Zoro, *One Piece*

Examples

standard input	standard output
1 0 0 3 1 1 0 3 1 2 2 3 4 0 0 3 4 0 1 3 1 3 2 3 4 2 0 3 4 3 1 3 4 2 3 3 0	0 1 0 1 0 2 1 0 2 0 1 2 0 1 3 2 3 0 1 2 3 2 0 1 3 2 1 0
1 0 0 1 1 1 1 2 2 1 3 3 3 5 3 1 4 0 5 3 0	0 1 2 3 3 2 1 0 0 1 2 3 4
1 0 0 1 1 1 1 2 2 1 3 3 3 7 0 3 7 1 3 6 1 3 6 2 3 1 4 2 3 6 4 3 7 3 3 0	0 1 2 3 0 1 2 3 3 0 1 2 0 1 2 3 2 3 0 1 2 3 4 0 1 1 2 3 4 0 3 4 0 1 2